

Student Performance EDA

Complete Code Guide with Line-by-Line Explanations

This document covers every code group from the Student Performance Exploratory Data Analysis project. Each section presents the code block followed by a detailed explanation of every line — what it does, why it is used, and how it contributes to the overall analysis pipeline.

Step 1

Importing Libraries

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
import warnings
warnings.filterwarnings('ignore')
```

import numpy as np

NumPy provides fast numerical array operations. Aliased as np. Used for utilities like np.arange() when computing bar positions in grouped bar charts.

import pandas as pd

Pandas is the core data library. Aliased as pd. Nearly every operation — loading, filtering, grouping, aggregating — runs through a Pandas DataFrame.

import seaborn as sns

Seaborn is a high-level visualization library built on Matplotlib. Aliased as sns. Produces statistical plots (histplots, barplots, violin plots) with minimal code.

import matplotlib.pyplot as plt

Matplotlib's plotting interface, aliased as plt. Controls figure size, subplot layout, axis labels, titles, and the plt.show() call that renders figures.

%matplotlib inline

A Jupyter magic command. Embeds plots directly inside notebook cells instead of opening a separate window.

import warnings

Python's built-in warnings module, imported so we can control how warnings are displayed in the output.

warnings.filterwarnings('ignore')

Suppresses all warnings (e.g. library deprecation notices). Keeps the notebook output clean and focused on results.

Step 2

Loading the Dataset

```
df = pd.read_csv('data/stud.csv')  
df.head()  
df.shape
```

df = pd.read_csv('data/stud.csv')	Reads the CSV file from disk and stores it as a Pandas DataFrame named df. A DataFrame is a 2D table — rows are students, columns are their attributes.
df.head()	Displays the first 5 rows. Provides a quick visual preview of column names, sample values, and overall data structure before any analysis begins.
df.shape	Returns a tuple (rows, columns). Confirms the dataset has 1000 students and 8 columns — a quick sanity check that all data loaded correctly.

Insight

The dataset contains 1000 rows and 8 columns: 5 categorical (gender, ethnicity, parental education, lunch, test prep) and 3 numeric (math, reading, writing scores).

Step 3**Data Quality Checks**

```
df.isna().sum()  
df.duplicated().sum()  
df.info()  
df.nunique()  
df.describe()
```

df.isna().sum()

isna() creates a True/False mask for every cell — True = missing. .sum() counts the True values per column. Non-zero counts indicate columns that need imputation or removal.

df.duplicated().sum()

Checks whether any row is an exact copy of another. .sum() gives the total duplicate count. Duplicates skew statistics and model training if left in.

df.info()

Prints a concise summary: total rows, each column's name, non-null count, and dtype (int64 for numbers, object for strings). Essential for spotting type mismatches early.

df.nunique()

Returns the number of distinct values per column. Reveals cardinality — gender has 2, race_ethnicity has 5, math_score has 81 unique values.

df.describe()

Computes descriptive statistics for numeric columns: count, mean, std, min, quartiles (25/50/75%), and max. Reveals distribution shape and potential extreme values.

Insight

Result: zero missing values, zero duplicates. Score means range 66-69 with std ~15. Minimum math score is 0 — a notable outlier to investigate in visualizations.

Step 4

Exploring Categories & Separating Column Types

```
print(df['gender'].unique())
print(df['race_ethnicity'].unique())
print(df['parental_level_of_education'].unique())
print(df['lunch'].unique())
print(df['test_preparation_course'].unique())

numeric_features = [f for f in df.columns if df[f].dtype != 'O']
categorical_features = [f for f in df.columns if df[f].dtype == 'O']
```

df['gender'].unique()	Returns all distinct values in the gender column as an array — ['female', 'male']. Repeated for each categorical column to detect typos, extra whitespace, or unexpected categories.
numeric_features = [... dtype != 'O']	A list comprehension that keeps only columns whose dtype is NOT 'O' (Object/text). Result: ['math_score', 'reading_score', 'writing_score'] — the three numeric score columns.
categorical_features = [... dtype == 'O']	Same pattern but keeps dtype == 'O' columns. Result: the 5 demographic columns. Separating types is critical — mean/std only apply to numbers; value_counts only applies to categories.

Insight

Separating numeric and categorical columns prevents type errors and guides which analysis methods to apply. You can compute a mean on scores but not on text like 'female' or 'Group C'.

Step 5

Feature Engineering: Total Score & Average

```
df['total score'] = df['math_score'] + df['reading_score'] + df['writing_score']
df['average'] = df['total score'] / 3
```

df['total score'] = df['math_score'] + ...	Creates a new column by adding three subject scores for each row. Pandas performs element-wise arithmetic automatically — no loop needed. Each student receives their own summed value.
df['average'] = df['total score'] / 3	Creates another new column by dividing each student's total by 3. This single metric represents overall performance, making comparisons across groups and visualizations much simpler.

Insight

Feature engineering — creating new informative columns from existing ones — is a core data science step. One 'average' column is far easier to analyze than three separate scores in parallel.

Step 6**Counting Top Scorers & Low Scorers**

```
math_full = df[df['math_score'] == 100]['average'].count()
writing_full = df[df['writing_score'] == 100]['average'].count()
reading_full = df[df['reading_score'] == 100]['average'].count()

math_less_20 = df[df['math_score'] <= 20]['average'].count()
writing_less_20 = df[df['writing_score'] <= 20]['average'].count()
reading_less_20 = df[df['reading_score'] <= 20]['average'].count()
```

df[df['math_score'] == 100]

Boolean indexing: the inner condition creates a True/False Series. Passing it into df[...] filters to only the rows where that condition is True — students with a perfect math score.

['average'].count()

After filtering, selects the 'average' column and counts non-null entries. Since there are no missing values, this equals the number of students who passed the filter condition.

df[df['math_score'] <= 20]

Same Boolean indexing but with the <= operator (less than or equal). Isolates students scoring 20 or below — the weakest performers. The pattern is identical; only the threshold changes.

Insight

Reading has the most perfect scores (17) and fewest low scores (1). Maths has the worst extremes: only 7 perfect scores and 4 students at 20 or below — confirming it is the hardest subject.

Step 7**Histogram & KDE Plots**

```
fig, axs = plt.subplots(1, 2, figsize=(15, 7))
plt.subplot(121)
sns.histplot(data=df, x='average', bins=30, kde=True, color='g')
plt.subplot(122)
sns.histplot(data=df, x='average', kde=True, hue='gender')
plt.show()
```

plt.subplots(1, 2, figsize=(15, 7))

Creates a figure with 1 row and 2 subplot columns. `figsize=(15, 7)` sets total width to 15 inches and height to 7 — prevents charts from appearing cramped or overlapping.

plt.subplot(121)

Activates subplot position 1 of a 1-row, 2-column grid. The 3-digit shorthand 121 = 1 row, 2 columns, position 1. All subsequent plot commands render into this panel.

sns.histplot(..., bins=30, kde=True, color='g')

Draws a histogram of the 'average' column split into 30 bins. `kde=True` overlays a smooth Kernel Density Estimation curve showing the continuous distribution shape. `color='g'` = green bars.

plt.subplot(122)

Switches the active panel to position 2 (second column). Subsequent plot commands now target the right-hand subplot.

hue='gender'

Colors histogram bars by the gender column — one color per category. Enables direct comparison of male and female score distributions on the same chart without creating two separate figures.

plt.show()

Renders and displays the completed figure. Required to ensure the plot appears in all environments, including scripts run outside Jupyter.

Insight

The gender-split histogram reveals female students' distribution is shifted slightly right (higher scores). The overall distribution is approximately normal, peaking around 65-70.

Step 8

Gender Bivariate Analysis — Grouped Bar Chart

```
gender_group = df.groupby('gender').mean()
X = ['Total Average', 'Math Average']
female_scores = [gender_group['average'][0], gender_group['math_score'][0]]
male_scores = [gender_group['average'][1], gender_group['math_score'][1]]
X_axis = np.arange(len(X))
plt.bar(X_axis - 0.2, male_scores, 0.4, label='Male')
plt.bar(X_axis + 0.2, female_scores, 0.4, label='Female')
plt.xticks(X_axis, X)
plt.ylabel('Marks')
plt.legend()
plt.show()
```

<code>df.groupby('gender').mean()</code>	Groups the DataFrame by gender and computes the mean of every numeric column per group. Returns a small table with one row per gender showing average scores across all subjects.
<code>female_scores = [gender_group['average'][0], ...]</code>	Extracts the mean 'average' and 'math_score' for females (index 0 — female comes first alphabetically after groupby). Collected into a list to use as bar heights.
<code>X_axis = np.arange(len(X))</code>	Creates NumPy array [0, 1] — numeric positions for the two metric groups on the x-axis. Required to offset male and female bars precisely without overlap.
<code>plt.bar(X_axis - 0.2, male_scores, 0.4, label='Male')</code>	Draws male bars shifted 0.2 units left of each x position with bar width 0.4. The -0.2 offset creates space for the female bar to sit beside it.
<code>plt.bar(X_axis + 0.2, female_scores, 0.4, label='Female')</code>	Draws female bars shifted 0.2 units right. Together they form a grouped bar chart — two bars per metric, visually comparable side by side.
<code>plt.xticks(X_axis, X)</code>	Replaces numeric x-axis ticks (0, 1) with the readable labels 'Total Average' and 'Math Average' from the X list.
<code>plt.legend()</code>	Adds a legend box mapping bar colors to 'Male' and 'Female' labels — making the chart self-explanatory without needing annotation.

Insight

Females outperform males in overall average score, but males score higher specifically in maths. This nuance is only visible when comparing both metrics simultaneously in one chart.

Step 9

Race / Ethnicity Bivariate Analysis

```
Group_data2 = df.groupby('race/ethnicity')
f, ax = plt.subplots(1, 3, figsize=(20, 8))
sns.barplot(x=Group_data2['math_score'].mean().index,
y=Group_data2['math_score'].mean().values,
palette='mako', ax=ax[0])
ax[0].set_title('Math score')
for container in ax[0].containers:
ax[0].bar_label(container, color='black', size=15)
```

df.groupby('race/ethnicity')

Groups the DataFrame by each ethnic group (A through E). Creates a GroupBy object — no computation yet; just the grouping structure primed for aggregation calls.

Group_data2['math_score'].mean().index

Computes mean math score per group, then extracts the group names (Group A–E) as the x-axis labels for the barplot.

Group_data2['math_score'].mean().values

Extracts the computed mean scores as a NumPy array. These become the bar heights on the y-axis.

palette='mako'

Applies Seaborn's 'mako' sequential color palette — shades from light to dark — visually differentiating the 5 groups without implying a value ranking.

ax=ax[0]

Directs this barplot into the first subplot panel (index 0) of the 3-panel figure. Without this, Seaborn would create a new figure and the layout would break.

**for container in ax[0].containers:
bar_label(...)**

Loops over each bar object and places its numeric value directly above the bar. Eliminates the need to read precise heights from the y-axis scale.

Insight

Group E scores highest across all three subjects; Group A scores lowest. This pattern holds for both genders — suggesting a socioeconomic correlation with academic performance.

Violin Plots, Pie Charts & Outlier Detection (Box Plots)

```
# Violin plot – score distribution shape
sns.violinplot(y='math_score', data=df, color='red', linewidth=3)

# Pie chart – categorical breakdown
plt.pie(size, colors=color, labels=labels, autopct='%.2f%%')

# Box plot – outlier detection
sns.boxplot(df['math_score'], color='skyblue')
```

sns.violinplot(y='math_score', ...)	A violin plot is a KDE rotated and mirrored into a symmetrical shape. Width at any vertical point represents density — a bulge means many students scored there. Combines distribution shape with summary statistics.
linewidth=3	Sets the border line of the violin to 3pt thickness. Makes the outline clearly visible even when multiple violins are displayed side by side in a figure.
plt.pie(size, autopct='%.2f%%')	Draws a pie chart where size = value_counts() of a categorical column. autopct='%.2f%%' formats each slice label as a 2-decimal percentage — e.g., '32.10%'.
sns.boxplot(df['math_score'], color='skyblue')	A box plot shows median (center line), IQR (the box from Q1 to Q3), whiskers (1.5x IQR), and outliers as individual dots. Instantly identifies extreme scores that may distort analysis.

Insight

Most students score 60-80 in maths and 50-80 in reading and writing — visible from violin plot widths. Box plots confirm a small cluster of extreme outliers near 0 in maths.

Pairplot — Multivariate Correlation Analysis

```
sns.pairplot(df, hue='gender')  
plt.show()
```

sns.pairplot(df, hue='gender')	Automatically generates a grid of scatter plots for every pair of numeric columns — math vs reading, math vs writing, reading vs writing — and KDE plots along the diagonal. hue='gender' colors each point by gender, revealing group-level patterns across all relationships at once.
plt.show()	Renders the complete pairplot grid. Since pairplot creates many internal subplots, an explicit plt.show() ensures the full multi-panel figure is displayed correctly.

Insight

The pairplot confirms strong positive linear correlation between all three scores — students who excel in one subject tend to excel in all three. Female students cluster slightly higher in reading and writing.

Quick Reference Summary

Step	Operation	Key Functions	Purpose
1	Import Libraries	import, warnings.filterwarnings	Load all tools for analysis
2	Load Dataset	pd.read_csv(), head(), shape	Read CSV, preview structure
3	Data Quality Checks	isna(), duplicated(), info(), describe(), nunique()	Verify data is clean
4	Explore Categories	unique(), list comprehension	Identify distinct values; split column types
5	Feature Engineering	Column arithmetic (+, /)	Create total_score and average columns
6	Top / Low Scorer Count	Boolean indexing, .count()	Count perfect and failing students per subject
7	Histogram & KDE	plt.subplots(), sns.histplot()	Visualize score distribution by gender
8	Gender Bar Chart	groupby().mean(), plt.bar()	Compare male vs female on two metrics
9	Ethnicity Bar Chart	groupby(), sns.barplot(), bar_label()	Compare scores across 5 ethnic groups
10	Violin / Pie / Box Plots	violinplot(), pie(), boxplot()	Distribution shape, category breakdown, outliers
11	Pairplot	sns.pairplot()	Correlate all numeric variables at once